

TASK 9

DOCKER FUNDAMENTALS AND CONTAINER LIFECYCLE MANAGEMENT

Practical Implementation Report

Student Name	Saikumar Padala
Roll Number	15
PRN / Student ID	5711387
Tools	Docker Desktop, Docker CLI, Windows PowerShell

Academic Practical Report

TABLE OF CONTENTS

Section	Title
1	Objective
2	Tools and Environment
3	Introduction to Docker
4	Docker Architecture
5	Practical Implementation
5.1	Engine and Version Verification
5.2	Images and Containers
5.3	Custom Dockerfile and Image Build
5.4	Inspection and Logs
5.5	Nginx Web Server
5.6	Docker Networking
5.7	Volume Persistence
5.8	Docker Desktop Dashboard and Transcript
6	Results and Observations
7	Conclusion
Appendix A	Command Log Extract

1. OBJECTIVE

To understand Docker architecture and manage containerized applications by working with images, containers, lifecycle operations, networking, and persistent storage. The practical also demonstrates building and running a custom image from a Dockerfile.

2. TOOLS AND ENVIRONMENT

The practical was performed on Windows using Windows PowerShell and Docker Desktop. The uploaded transcript identifies Windows PowerShell 5.1. The screenshots show Docker CLI commands using the desktop-linux context and Docker Desktop reporting “Engine running.”

3. INTRODUCTION TO DOCKER

Docker is a containerization platform that packages an application and its dependencies into an image. A container is a runnable instance of an image. Images can be downloaded from a registry or built locally. Containers can be inspected, stopped, started, and removed independently of the images from which they were created.

4. DOCKER ARCHITECTURE

Docker follows a client-server model. The Docker CLI sends requests to the Docker Engine API. The engine manages images, containers, networks, and volumes. Registries store and distribute images. Docker Desktop provides the Windows desktop interface and the Linux container backend used for this practical.

Component	Purpose
Docker CLI	Accepts commands such as docker build, docker run, docker images, and docker inspect.
Docker Engine	Manages container execution and Docker resources.
Image registry	Stores images that can be pulled to a local environment.
Dockerfile	Defines repeatable image-building instructions.
Volumes and networks	Provide persistent storage and container connectivity.

5. PRACTICAL IMPLEMENTATION

5.1 Docker Engine and Version Verification

```
docker --version
docker info
```

The screenshot shows Docker version 29.8.0 and server information for the desktop-linux context. Docker Desktop's status bar reports that the engine is running.

5.2 Images and Containers

```
docker images
docker ps -a
```

The image listing includes hello-world, nginx, alpine, PostgreSQL images, and task9-python-demo:latest. The container listing shows the hello-world and custom Python test containers in an exited state after their commands completed.

5.3 Custom Dockerfile and Image Build

A simple Python image was built to demonstrate the Dockerfile-to-image-to-container workflow. Dockerfile contents:

```
FROM python:3.12-slim

WORKDIR /app

CMD ["python", "-c", "print('Task 9: Docker container is running successfully!')"]
```

Build and run commands:

```
docker build -t task9-python-demo .
docker run --name task9-python-container task9-python-demo
```

The screenshot shows a successful image build and the output “Task 9: Docker container is running successfully!” The image was tagged as task9-python-demo:latest.

5.4 Container Inspection and Logs

```
docker inspect task9-python-container
docker logs task9-python-container
```

The submitted screenshot shows docker inspect output including container identity, image, command, runtime settings, and network mode. The custom container exited after printing its message. The uploaded transcript lists inspection and log commands, but does not preserve their output text.

5.5 Nginx Web Server and Port Mapping

```
docker run -d --name task9-web -p 8080:80 nginx
```

The browser screenshot displays the default “Welcome to nginx!” page at `http://localhost:8080`. This demonstrates that the web server was reachable through host port 8080 mapped to container port 80.

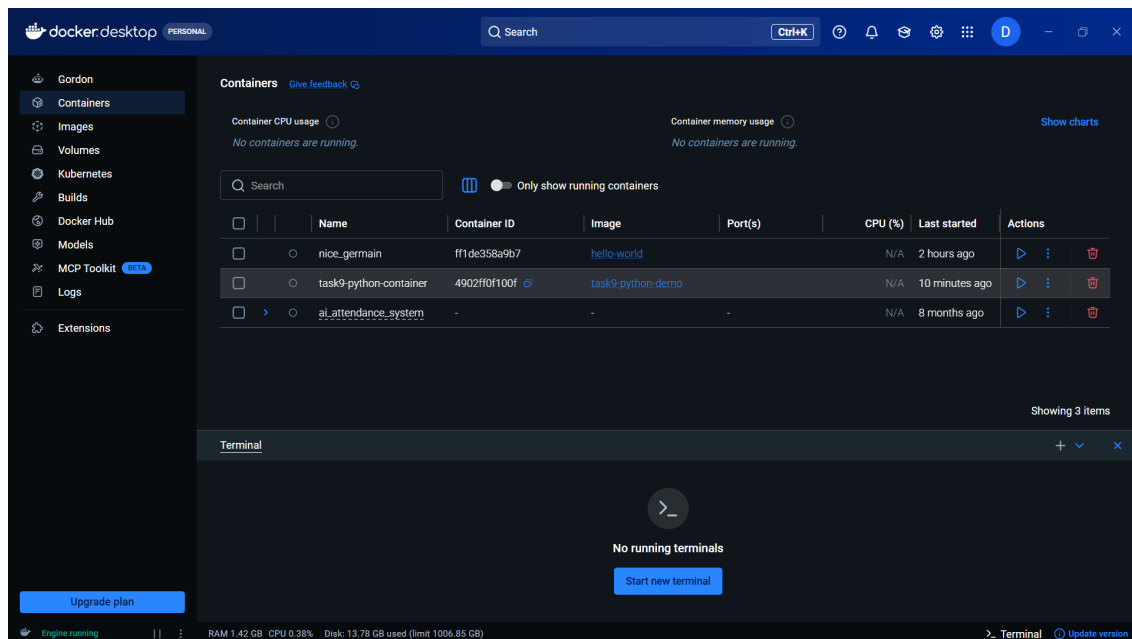


Figure 1. Nginx welcome page opened at localhost:8080.

5.6 Docker Networking

```
docker network ls
docker network inspect bridge
```

The screenshots show the default bridge, host, and none networks. The bridge inspection output includes its subnet and gateway configuration.

5.7 Docker Volume Persistence

```
docker volume create task9-data
docker run --rm -v task9-data:/data alpine sh -c "echo 'Docker volume persistence test' > /data/test.txt"
docker run --rm -v task9-data:/data alpine cat /data/test.txt
```

The screenshot shows creation of the task9-data named volume and reading back the text “Docker volume persistence test” from a subsequent temporary Alpine container. This demonstrates persistence beyond the lifetime of the first temporary container.

```
C:\Windows\System32\cmd.exe X Windows PowerShell X + - O X
PS C:\Users\padal\Downloads\Task9_Docker_Practical> docker --version
Docker version 20.10.8, build 00000000
PS C:\Users\padal\Downloads\Task9_Docker_Practical> docker info
Client:
Version: 20.10.8
Context: desktop-linux
Debug Mode: false
Plugins:
Agent: Docker AI Agent Runner (Docker Inc.)
Version: v1.108.8
Path: C:\Program Files\Docker\cli-plugins\docker-agent.exe
ai: Docker AI Agent - Ask Gordon (Docker Inc.)
Version: v1.11.0
Path: C:\Program Files\Docker\cli-plugins\docker-ai.exe
Buildx: Docker Buildx (Docker Inc.)
Version: v0.17.1
Path: C:\Program Files\Docker\cli-plugins\docker-buildx.exe
compose: Docker Compose (Docker Inc.)
Version: v2.1.1
Path: C:\Program Files\Docker\cli-plugins\docker-compose.exe
debug: Get a shell into any image or container (Docker Inc.)
Version: 0.6.0
Path: C:\Program Files\Docker\cli-plugins\docker-debug.exe
desktop: Docker Desktop commands (Docker Inc.)
Version: v0.4.4
Path: C:\Program Files\Docker\cli-plugins\docker-desktop.exe
dhl: CLI for managing Docker Hardened Images (Docker Inc.)
Version: v0.0.0
Path: C:\Program Files\Docker\cli-plugins\docker-dhl.exe
extension: Manages Docker extensions (Docker Inc.)
Version: v0.2.1
Path: C:\Program Files\Docker\cli-plugins\docker-extension.exe
init: Creates Docker-related starter files for your project (Docker Inc.)
Version: v1.0.0
Path: C:\Program Files\Docker\cli-plugins\docker-init.exe
mcp: Docker MCP Plugin (Docker Inc.)
Version: v0.0.3
Path: C:\Program Files\Docker\cli-plugins\docker-mcp.exe
model: Docker Model Runner (Docker Inc.)
Version: v1.2.6
Path: C:\Program Files\Docker\cli-plugins\docker-model.exe
offload: Docker Offload (Docker Inc.)
Version: v0.0.27
Path: C:\Program Files\Docker\cli-plugins\docker-offload.exe
pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
Version: v0.2.2
Path: C:\Program Files\Docker\cli-plugins\docker-pass.exe
sandbox: Docker Sandbox is deprecated, use Docker Sandboxes instead (Docker Inc.)
Version: v0.11.0
Path: C:\Program Files\Docker\cli-plugins\docker-sandbox.exe
scout: Docker Scout (Docker Inc.)
Version: v1.24.0
Path: C:\Program Files\Docker\cli-plugins\docker-scout.exe
Server:
Containers: 3
Running: 0
Paused: 0
Stopped: 0
Images: 9
Server Version: 20.10.8
Storage Driver: overlayfs
Driver Type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: cgroupfs
Cgroup Version: 2
Plugins:
```

Figure 2. PowerShell evidence for Docker networking and named-volume persistence.

5.8 Docker Desktop Dashboard

The Docker Desktop Containers view shows task9-python-container associated with task9-python-demo and reports that no containers are running. This is consistent with the one-shot Python container having completed its command and exited.

```
C:\Windows\System32\cmd.exe X + - O X
(c) Microsoft Corporation. All rights reserved.
C:\Users\padal\Downloads\Task9_Docker_Practical> docker build -t task9-python-demo .
[*] Building 0.8s (1/1) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 2B
ERROR: failed to build: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/6x1kjpepwh3djoglamx2txr8f
C:\Users\padal\Downloads\Task9_Docker_Practical>
C:\Users\padal\Downloads\Task9_Docker_Practical> docker build -t task9-python-demo .
[*] Building 11.0s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 164B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/2] FROM docker.io/library/python:3.12-slim@sha256:ddd47e07f9d15aeca61529320492139d21cac7f0070c08609243e51e4e0016
=> => resolve docker.io/library/python:3.12-slim@sha256:ddd47e07f9d15aeca61529320492139d21cac7f0070c08609243e51e4e0016
=> sha256:5e62c0470a0972ad9561040c8a37908c3252eda1dd19e08ca7e76a37b02b5dc7 240B / 240B
=> sha256:e360d85fde0f3910a4fe48ba2bb0865d5e618392ea8bfa424e66875e45273c0 12.12MB / 12.12MB
=> sha256:a510a9ea6d60786b7cefd8f9577945344c72879881de8d32ef07ec38111fb045 4.27MB / 4.27MB
=> => extracting sha256:a510a9ea6d60786b7cefd8f9577945344c72879881de8d32ef07ec38111fb045
=> => extracting sha256:e360d85fde0f3910a4fe48ba2bb0865d5e618392ea8bfa424e66875e45273c0
=> => extracting sha256:5e62c0470a0972ad9561040c8a37908c3252eda1dd19e08ca7e76a37b02b5dc7
=> [2/2] WORKDIR /app
=> => exporting to image
=> => exporting manifest sha256:a6d643065c768250c36be30dde60c07e1b708dd41b9fb82166673b1e1d1476b7
=> => exporting config sha256:e5fe8561696ecb24efc8421a3c090d444b5cd5b91cd0c8d427beca2e7b61b55
=> => exporting attestation manifest sha256:4912fc58d9bd1348bb15914cfa2654b50855d2ee515bb15d35ffe0ddc75e82c4
=> => exporting manifest list sha256:e53d6ee8559bafede0e8c24db37809026088517bb5e4d62207f98a844f599e17
=> => naming to docker.io/library/task9-python-demo:latest
=> => unpacking to docker.io/library/task9-python-demo:latest
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/70cpxkxa8bhefun6jfhcr7a0f
C:\Users\padal\Downloads\Task9_Docker_Practical> docker run --name task9-python-container task9-python-demo
Task 9: Docker container is running successfully!
C:\Users\padal\Downloads\Task9_Docker_Practical>
```

Figure 3. Docker Desktop Containers dashboard showing the custom image and container.

6. RESULTS AND OBSERVATIONS

Test	Observation	Status
Engine verification	Docker version and server information displayed; Docker Desktop engine running.	Verified
Custom image build/run	task9-python-demo built and printed the expected success message.	Verified
Container inspection	Container configuration visible in inspect output; container exited after execution.	Verified
Nginx	Default Nginx page shown in browser at localhost:8080.	Verified
Networking	Default networks listed and bridge configuration inspected.	Verified
Volume persistence	Text written to a named volume was read back from another temporary container.	Verified
Command transcript	Commands are recorded, but output sections are blank in the uploaded transcript.	Partial log

Key observations

1. Dockerfiles support repeatable image builds.
2. Containers can exit normally after completing their configured command.
3. Port publishing allows a containerized web service to be reached from a host browser.
4. Named volumes keep data separate from an individual container's lifecycle.
5. Docker network inspection helps reveal network addressing and configuration.

7. CONCLUSION

The practical demonstrated core Docker operations: verifying the engine, listing images and containers, building a custom Python image, running and inspecting a container, accessing Nginx through host port mapping, inspecting Docker networks, and verifying named-volume persistence. The screenshots support these observed outcomes. The command transcript is included as supporting documentation, with the limitation that command output was not captured in the text file.

APPENDIX A. DOCKER COMMAND LOG EXTRACT

The uploaded PowerShell transcript contains two transcript sessions and the command entries below. Its output sections are blank, so the appendix lists commands only and does not claim that the text file independently verifies their results.

```
docker --version
docker images
docker ps -a
docker logs task9-python-container
docker inspect task9-python-container
docker network ls
docker network inspect bridge
docker volume ls
docker run --rm -v task9-data:/data alpine cat /data/test.txt
Stop-Transcript
docker info
```

The unique command list is included for readability; repeated transcript sessions are not reproduced verbatim.